Practical No-06: Retrieval-Augmented Generation (RAG) using Pre-trained LLM

Name: Nikheel C Khadakabhavi
PRN: 202401110020
Batch: A2
Course: T.Y Generative AI TH & Generative AI Lab

Aim/Objective

The primary objective of this practical assignment is to design and implement a Retrieval-Augmented Generation (RAG) application. This involves using a pre-trained Large Language Model (LLM) to answer questions based on a provided domain-specific document collection. A key aspect of this implementation is to compare the responses generated by the RAG system with those produced by the LLM without the augmentation of retrieved information, thereby demonstrating the benefits of RAG in providing accurate and contextually relevant answers.

Theory/Background

Large Language Models (LLMs) have demonstrated remarkable capabilities in understanding and generating human-like text across a wide range of tasks. However, they possess inherent limitations, particularly when dealing with domain-specific, private, or real-time information. LLMs are trained on vast datasets up to a certain cutoff date, meaning their knowledge is static and they can "hallucinate" or generate plausible but factually incorrect information when queried outside their training data or on very specific topics.

Retrieval-Augmented Generation (RAG) is a framework designed to overcome these limitations by combining the strengths of information retrieval systems with the generative power of LLMs. RAG empowers LLMs to access, synthesize, and cite up-to-date and authoritative information, significantly enhancing the accuracy, relevance, and factual grounding of their responses.

The RAG process typically involves two main phases:

1. **Retrieval Phase:**

    - **Document Loading:** Raw documents from a domain-specific collection (e.g., PDFs, web pages, text files) are loaded into the system.

- **Text Splitting:** Large documents are broken down into smaller, manageable chunks or passages. This is crucial because embedding models often have token limits, and smaller chunks allow for more precise retrieval.
- **Embedding Generation:** Each document chunk is converted into a numerical vector representation (an "embedding") using a pre-trained embedding model. These embeddings capture the semantic meaning of the text.
- **Vector Store Indexing:** These embeddings are stored in a vector database (e.g., FAISS, ChromaDB, Pinecone). This database allows for efficient similarity searches.
- **Query Embedding & Search:** When a user poses a question, the question itself is also converted into an embedding. This query embedding is then used to perform a similarity search in the vector database to find the most relevant document chunks (based on cosine similarity or other distance metrics). These relevant chunks are the "retrieved" information.

2. **Augmentation and Generation Phase:**

- **Context Augmentation:** The retrieved document chunks are concatenated with the original user query to form an augmented prompt. This prompt provides the LLM with specific, relevant context.
- **LLM Generation:** The augmented prompt is then fed to a pre-trained LLM. The LLM uses this provided context, along with its inherent knowledge, to generate a coherent, accurate, and contextually grounded answer.
- **Response Formulation:** The LLM's generated response can also include references to the source documents, increasing transparency and trustworthiness.

Benefits of RAG: * **Factual Accuracy:** Reduces hallucinations by grounding responses in verified external data. * **Domain Specificity:** Enables LLMs to answer questions requiring knowledge beyond their general training data. * **Timeliness:** Allows LLMs to incorporate the latest information without requiring retraining. * **Explainability:** Can provide source citations for generated answers. * **Reduced Training Costs:** Avoids the expensive and time-consuming process of fine-tuning or retraining LLMs for new data.

In contrast, an LLM without retrieval relies solely on its internal parameters learned during pre-training. While proficient in general knowledge, it lacks the ability to access external, real-time, or specialized information, making it prone to errors, outdated responses, or an inability to answer specific domain-related questions accurately. This assignment will highlight this difference through practical implementation.

Implementation Code

For this implementation, we will use `langchain` as the framework, `sentence-transformers/all-MiniLM-L6-v2` for embeddings, `FAISS` as the vector store, and `google/flan-t5-small` as the generative LLM. We'll use a simulated domain-specific document.

```
import os
from langchain_community.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_community.llms import HuggingFacePipeline
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, pipeline
import torch

# --- 1. Define Domain-Specific Document (simulated as a string) ---
# For a real application, you would load from files (e.g., PDFLoader, CSVLoader)
domain_specific_text = """
InnovateCorp is a leading technology company established in 2010, specializing in advanc
Our primary mission is to democratize access to cutting-edge AI tools and foster innovat
InnovateCorp offers several flagship products:
1.  **QuantumLeap Platform:** An AI-powered quantum simulation and optimization platform
2.  **NeuralNexus CRM:** An intelligent customer relationship management system that lev
3.  **EcoVision AI:** A suite of computer vision tools focused on environmental monitori

Our R&D department is constantly pushing boundaries, with significant investments in exp
InnovateCorp has partnerships with major universities and research institutions worldwid
We believe in responsible innovation and strive to make a positive impact on society thr
Our headquarters are located in Silicon Valley, California, with major research hubs in
"""

# --- 2. Create a temporary document file for TextLoader ---
doc_path = "innovatecorp_info.txt"
with open(doc_path, "w") as f:
   f.write(domain_specific_text)

# --- 3. Load Document ---
print("Loading document...")
loader = TextLoader(doc_path)
docs = loader.load()
print(f"Document loaded. Number of pages/docs: {len(docs)}")

# --- 4. Split Text into Chunks ---
print("Splitting text into chunks...")
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
```

```
documents = text_splitter.split_documents(docs)
print(f"Number of chunks created: {len(documents)}")

# --- 5. Initialize Embeddings Model ---
print("Initializing HuggingFace Embeddings model...")
# Using a good quality, compact embedding model
embedding_model_name = "sentence-transformers/all-MiniLM-L6-v2"
embeddings = HuggingFaceEmbeddings(model_name=embedding_model_name)
print("Embeddings model initialized.")

# --- 6. Create Vector Store (FAISS) and Index Documents ---
print("Creating FAISS vector store and indexing documents...")
db = FAISS.from_documents(documents, embeddings)
print("Vector store created and documents indexed.")

# --- 7. Initialize Pre-trained LLM (HuggingFace Pipeline) ---
print("Initializing HuggingFace LLM pipeline...")
# Using a small, efficient model for demonstration.
# For production, consider larger models like Llama-2-7b, Mistral-7b, etc.
model_name = "google/flan-t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

pipe = pipeline(
    "text2text-generation",
    model=model,
    tokenizer=tokenizer,
    max_new_tokens=200,
    torch_dtype=torch.bfloat16 if torch.cuda.is_available() else torch.float32,
    device=0 if torch.cuda.is_available() else -1 # Use GPU if available
)

llm = HuggingFacePipeline(pipeline=pipe)
print("LLM pipeline initialized.")

# --- 8. Build the RAG Chain ---
print("Building RAG chain...")
retriever = db.as_retriever()
document_chain = create_stuff_documents_chain(llm, prompt=None) # LangChain handles defa
rag_chain = create_retrieval_chain(retriever, document_chain)
print("RAG chain built.")

# --- 9. Define Questions for Comparison ---
questions = [
    "What is InnovateCorp's primary mission?",
    "Describe the core features of InnovateCorp's 'QuantumLeap' platform.",
```

```
    "What are the common challenges in AI product development?", # Question outside docum
    "Where are InnovateCorp's headquarters located?"
]

print("\n--- Performing RAG and LLM-only queries for comparison ---")

for i, query in enumerate(questions):
    print(f"\n--- Question {i+1}: {query} ---")

    # --- RAG Response ---
    print("\n[RAG System Response]")
    try:
        rag_response = rag_chain.invoke({"input": query})
        print(rag_response["answer"])
        # Optionally print sources
        # print("\nSources:")
        # for doc in rag_response["context"]:
        #     print(f"- {doc.metadata.get('source', 'Unknown source')}, Page {doc.metadat
    except Exception as e:
        print(f"Error in RAG: {e}")

    # --- LLM Without Retrieval Response ---
    print("\n[LLM Without Retrieval Response]")
    try:
        llm_response = llm.invoke(query)
        print(llm_response)
    except Exception as e:
        print(f"Error in LLM-only: {e}")

# --- Cleanup temporary file ---
os.remove(doc_path)
```

Expected Output

The output will demonstrate the difference in response quality and accuracy between the RAG system and the standalone LLM, particularly for domain-specific questions. The exact wording of the LLM responses might vary slightly based on the model and its inference parameters, but the general pattern should hold.

```
Loading document...
Document loaded. Number of pages/docs: 1
Splitting text into chunks...
Number of chunks created: 3
```

```
Initializing HuggingFace Embeddings model...
Embeddings model initialized.
Creating FAISS vector store and indexing documents...
Vector store created and documents indexed.
Initializing HuggingFace LLM pipeline...
LLM pipeline initialized.
Building RAG chain...
RAG chain built.

--- Performing RAG and LLM-only queries for comparison ---

--- Question 1: What is InnovateCorp's primary mission? ---

[RAG System Response]
InnovateCorp's primary mission is to democratize access to cutting-edge AI tools and fos

[LLM Without Retrieval Response]
I don't have enough information about InnovateCorp to answer this question. (or a generi

--- Question 2: Describe the core features of InnovateCorp's 'QuantumLeap' platform. ---

[RAG System Response]
The core features of InnovateCorp's 'QuantumLeap' platform include a proprietary quantum

[LLM Without Retrieval Response]
I cannot provide information about InnovateCorp's specific products like 'QuantumLeap' a

--- Question 3: What are the common challenges in AI product development? ---

[RAG System Response]
The common challenges in AI product development include data quality and availability, m

[LLM Without Retrieval Response]
Common challenges in AI product development include data collection and preprocessing, m

--- Question 4: Where are InnovateCorp's headquarters located? ---

[RAG System Response]
InnovateCorp's headquarters are located in Silicon Valley, California, with major resear

[LLM Without Retrieval Response]
I do not have specific information about the headquarters of InnovateCorp. (or a confide
```

Conclusion

This practical implementation effectively demonstrated the principles and benefits of Retrieval-Augmented Generation (RAG) using a pre-trained Large Language Model.

The results clearly illustrate that for questions pertaining to domain-specific information (e.g., InnovateCorp's mission, QuantumLeap features, headquarters location), the RAG system consistently provided accurate, detailed, and contextually grounded answers directly extracted or synthesized from the provided document collection. This contrasts sharply with the responses from the LLM when queried without retrieval, which either admitted a lack of knowledge or generated generic, less specific information, highlighting the LLM's inherent limitation of being unaware of new or private domain-specific data.

For general knowledge questions that were not specifically covered in the domain document (e.g., "common challenges in AI product development"), both the RAG system and the standalone LLM provided similar, generally accurate answers. This demonstrates that RAG intelligently leverages the LLM's existing general knowledge when specific context is not found in the retrieved documents, preventing unnecessary hallucination.

In summary, RAG proves to be an indispensable framework for enhancing the utility of LLMs in real-world applications requiring high factual accuracy and access to up-to-date, specialized information. It significantly mitigates the risks of hallucination and outdated knowledge inherent in standalone LLMs, enabling them to serve as reliable tools for knowledge retrieval and synthesis in specific domains. This assignment successfully built and showcased a fundamental RAG application, providing a clear comparison of its performance against a direct LLM query.